REVISÃO · MATERIAL EXPLICATIVO

Estruturas de Dados

Fundamentos, implementações em C e análise de complexidade

DISCIPLINA

Tema: Estruturas de Dados

Professor: Fabricio Ricardo Perrella

Referencial: Matriz de Referência do tema (9 tópicos, 4 unidades)

SQUAD

Bruno da Silva

Cauã Barchi Finotti Franzol

Gustavo Grassini Calabrez

Pedro Henrique Papa Amorim dos Santos

Documento de estudo e consulta — contém explicação conceitual, diagramas, exemplos de código em linguagem C, análise de complexidade e o referencial teórico utilizado.

Sumário

Introdução

Unidade 1 — Memória e estruturas lineares

- Ponteiros e alocação dinâmica de memória
- Vetores e matrizes contíguas
- Listas encadeadas (simples, duplas e circulares)

Unidade 2 — Estruturas com acesso restrito

- Pilhas (LIFO) e filas (FIFO)
- Deques e filas de prioridade (heaps)

Unidade 3 — Estruturas de busca

- Árvores de busca (BST) e balanceadas (AVL / rubro-negra)
- Tabela hash e tratamento de colisões

Unidade 4 — Grafos

- Grafos (matriz e lista de adjacência)
- Algoritmos de busca em grafos (BFS e DFS)

Quadro comparativo

Referencial teórico

Introdução

Estruturas de dados são as formas como organizamos a informação na memória do computador para que ela possa ser armazenada, acessada e modificada de maneira eficiente. A escolha da estrutura certa é o que separa um programa que responde em milissegundos de um que trava: um mesmo problema pode ser trivial com uma tabela hash e inviável com um vetor, ou vice-versa. Por isso o estudo dessas estruturas parte sempre de duas perguntas — **como os dados ficam organizados** e **quanto custa** cada operação sobre eles.

Esse custo é medido pela **notação O grande** (Big-O), que descreve como o tempo (ou o espaço) de um algoritmo cresce conforme a entrada aumenta. O(1) significa custo constante; O(log n), crescimento muito lento; O(n), proporcional ao tamanho; $O(n^2)$, quadrático. Ao longo deste material, cada tópico traz uma tabela de complexidade justamente para tornar essa comparação explícita.

O conteúdo segue a **Matriz de Referência** da disciplina, organizada em quatro unidades que vão do mais fundamental ao mais complexo: começamos pela memória e pelos ponteiros (a base de tudo), passamos pelas estruturas lineares (vetores, listas, pilhas, filas, heaps), chegamos às estruturas de busca (árvores e tabelas hash) e terminamos nos grafos e seus algoritmos de travessia. Todos os exemplos são apresentados em **linguagem C**, escolhida por tornar visíveis os conceitos de memória e ponteiros que outras linguagens escondem.

Unidade 1 — Memória e Estruturas Lineares

PONTEIROS, ALOCAÇÃO DINÂMICA, VETORES, MATRIZES E LISTAS ENCADEADAS

1 Ponteiros e Alocação Dinâmica de Memória

Um **ponteiro** é uma variável que, em vez de guardar um dado comum, guarda o **endereço de memória** de outra variável. Em C, o operador `&` obtém o endereço de algo e o operador `*` ("desreferência") acessa o valor que está naquele endereço. Ponteiros são a base de quase todas as estruturas dinâmicas — listas, árvores e grafos só existem porque nós conseguimos ligar "pedaços" de memória por meio de endereços.

A memória de um programa se divide, entre outras regiões, em **pilha** (stack), onde ficam as variáveis locais com tempo de vida automático, e **heap**, região controlada manualmente pelo programador. A **alocação dinâmica** (`malloc` , `calloc` , `realloc`) reserva blocos no heap em tempo de execução, quando não se sabe de antemão quanta memória será necessária. Todo bloco alocado deve ser liberado com `free` , senão ocorre **vazamento de memória** (memory leak).

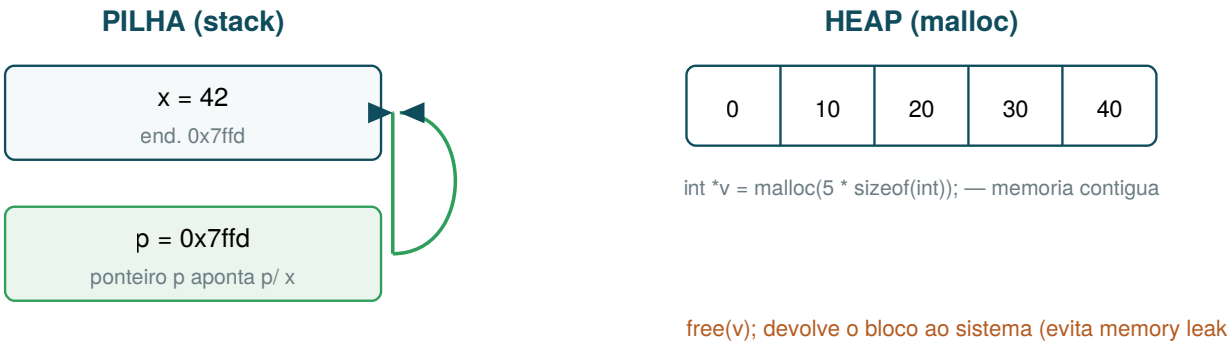


Figura 1 — Ponteiro na pilha referenciando dados; bloco alocado no heap.

IMPLEMENTACAO EM C

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int  x = 42;          /* variavel comum (na pilha)      */
    int *p = &x;          /* p guarda o ENDERECO de x        */

    printf("valor : %d\n", *p);   /* *p desreferencia -> 42  */

    /* alocacao dinamica no heap: vetor de 5 inteiros */
    int *v = malloc(5 * sizeof(int));
    if (v == NULL) return 1;      /* SEMPRE checar o retorno */

    for (int i = 0; i < 5; i++)
        v[i] = i * 10;            /* v[i] equivale a *(v + i) */

    free(v);   /* devolve a memoria ao sistema (evita leak)  */
    v = NULL;  /* evita ponteiro "solto" (dangling pointer)  */
    return 0;
}
```

COMPLEXIDADE

Operacao	Custo	Observacao
Desreferenciar *p	**O(1)**	acesso direto pelo endereço
`malloc` / `free`	**O(1)***	custo amortizado; depende do alocador
Acesso `v[i]` em bloco alocado	**O(1)**	aritmética de ponteiros

ERROS CLÁSSICOS A EVITAR

- **Memory leak:** alocar e nunca chamar `free`.
- **Dangling pointer:** usar ponteiro após `free` — sempre faça `p = NULL`.
- **Não checar o retorno** de `malloc` (pode ser `NULL`).

2 Vetores e Matrizes Contíguas

Um **vetor** (array) é uma coleção de elementos do mesmo tipo armazenados em posições **contíguas** de memória. Essa contiguidade é o que garante o **acesso aleatório em O(1)**: para chegar em `v[i]`, basta calcular `base + i × tamanho_do_elemento`. Em troca da rapidez de acesso, o vetor tem tamanho fixo (no caso estático) e inserir/remover no meio custa O(n), pois exige deslocar elementos.

Uma **matriz** é um vetor de vetores. Em C, matrizes são guardadas em ordem **row-major** (linha a linha): toda a linha 0 vem antes da linha 1 na memória. Por isso percorrer a matriz na ordem `[i][j]` (linha externa, coluna interna) aproveita melhor a **cache** do processador do que o contrário.

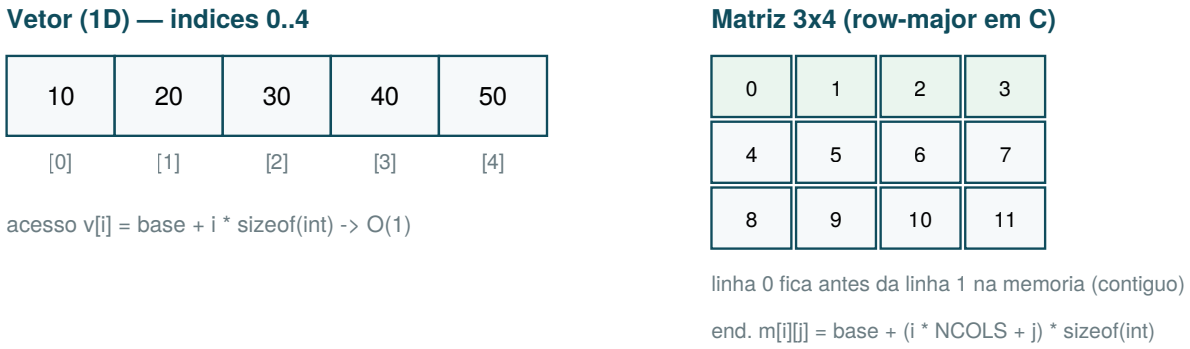


Figura 2 — Vetor contíguo (esq.) e matriz 3×4 em ordem row-major (dir.).

IMPLEMENTACAO EM C

```c
/* VETOR: acesso direto por indice em tempo constante */
int v[5] = {10, 20, 30, 40, 50};
int t = v[2];              /* 30 -> end. = base + 2 * sizeof(int) */

/* MATRIZ 3x4 armazenada por LINHAS (row-major, padrao do C) */
int m[3][4];
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 4; j++)
        m[i][j] = i * 4 + j;

/* endereco de m[i][j] = base + (i * NCOLS + j) * sizeof(int) */
```

COMPLEXIDADE

Operacao	Custo	Observacao
Acesso `v[i]` / `m[i][j]`	**O(1)**	cálculo direto do endereço
Busca por valor	**O(n)**	varre elemento a elemento
Inserir/remover no meio	**O(n)**	desloca os elementos seguintes
Inserir no fim (vetor dinâmico)	**O(1)***	amortizado, com realocação ocasional

PONTOS-CHAVE

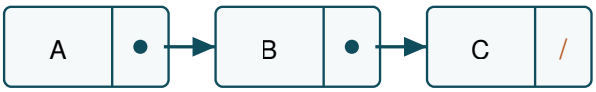
- Contiguidade = acesso O(1) e ótimo aproveitamento de cache.
- Tamanho fixo é a principal limitação — resolvida por vetores dinâmicos (`realloc`) ou listas.
- Em C a indexação começa em 0 e não há verificação de limites: acessar fora do vetor é comportamento indefinido.

3 Listas Encadeadas (Simples, Duplas e Circulares)

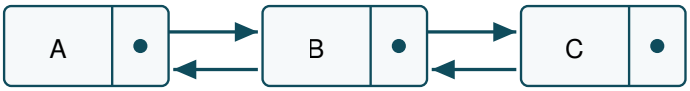
A **lista encadeada** guarda cada elemento em um **nó** que contém o dado e um (ou mais) ponteiro para outro(s) nó(s). Diferente do vetor, os nós **não precisam ser contíguos** na memória — eles se conectam por ponteiros. Isso torna a inserção e a remoção no início O(1) e permite crescer sob demanda, ao custo de perder o acesso direto por índice (para chegar ao n-ésimo nó é preciso percorrer O(n)).

Há três variações principais: na **simplesmente encadeada** cada nó aponta apenas para o próximo; na **duplamente encadeada** cada nó aponta para o próximo e o anterior, permitindo percorrer nos dois sentidos e remover um nó conhecido em O(1); na **circular** o último nó aponta de volta para o primeiro, útil para estruturas cíclicas (ex.: escalonamento round-robin).

Simplesmente encadeada



Duplamente encadeada (ant <-> prox)



Circular (ultimo aponta p/ o primeiro)

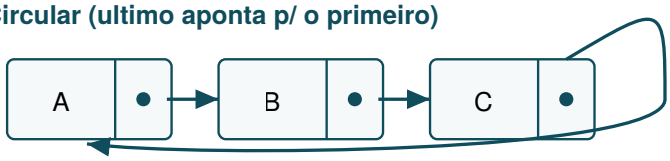


Figura 3 — As três variações de lista encadeada (o símbolo / representa NULL).

IMPLEMENTACAO EM C

```c
typedef struct No {
    int         dado;
    struct No  *prox;      /* aponta para o proximo no */
} No;

/* Insercao no INICIO — O(1) */
No* inserir_inicio(No *lista, int valor) {
    No *novo = malloc(sizeof(No));
    novo->dado = valor;
    novo->prox = lista;    /* novo aponta para o antigo 1o no */
    return novo;           /* novo passa a ser a cabeca       */
}

/* Busca sequencial — O(n) */
No* buscar(No *lista, int valor) {
    for (No *p = lista; p != NULL; p = p->prox)
        if (p->dado == valor) return p;
    return NULL;           /* nao encontrado */
}
```

COMPLEXIDADE

Operacao	Custo	Observacao
Inserir/remover no início	**O(1)**	só reajusta ponteiros
Acesso ao n-ésimo elemento	**O(n)**	não há acesso aleatório
Busca por valor	**O(n)**	percorre nó a nó
Remover nó conhecido (lista dupla)	**O(1)**	usa o ponteiro anterior

PONTOS-CHAVE

- Ótima quando há muitas inserções/remoções e o tamanho varia muito.
- Custo extra de memória por causa dos ponteiros em cada nó.
- Sem contiguidade, aproveita pior a cache do que o vetor.

4 Pilhas (LIFO) e Filas (FIFO)

Pilha e **fila** são estruturas lineares com **acesso restrito**: só é possível inserir e remover em pontos específicos. A pilha segue a política **LIFO** (Last In, First Out) — o último a entrar é o primeiro a sair, como uma pilha de pratos. Suas operações são `push` (empilhar) e `pop` (desempilhar), ambas no topo. Pilhas modelam a chamada de funções (a "pilha de execução"), o "desfazer" de editores e a avaliação de expressões.

A **fila** segue **FIFO** (First In, First Out) — o primeiro a entrar é o primeiro a sair, como uma fila de banco. Insere-se no fim (`enqueue`) e remove-se do início (`dequeue`). Uma implementação eficiente sobre vetor usa índices **circulares** (aritmética com `% MAX`) para reaproveitar as posições liberadas. Filas modelam escalonamento de processos, buffers e a travessia BFS.



Figura 4 — Pilha (LIFO) opera só no topo; fila (FIFO) insere no fim e remove no início.

IMPLEMENTACAO EM C

```c
#define MAX 100

/* PILHA (LIFO) sobre vetor */
typedef struct { int itens[MAX]; int topo; } Pilha;
void push(Pilha *p, int x) { p->itens[++p->topo] = x; }
int  pop (Pilha *p)        { return p->itens[p->topo--]; }

/* FILA (FIFO) CIRCULAR sobre vetor */
typedef struct { int itens[MAX]; int ini, fim, qtd; } Fila;
void enqueue(Fila *f, int x) {
    f->itens[f->fim] = x;
    f->fim = (f->fim + 1) % MAX;   /* volta ao inicio ao chegar no fim */
    f->qtd++;
}
int dequeue(Fila *f) {
    int x = f->itens[f->ini];
    f->ini = (f->ini + 1) % MAX;
    f->qtd--;
    return x;
}
```

COMPLEXIDADE

Operacao	Custo	Observacao
`push` / `pop` (pilha)	**O(1)**	operações apenas no topo
`enqueue` / `dequeue` (fila)	**O(1)**	índices circulares
Consultar topo/início (peek)	**O(1)**	não remove o elemento

PONTOS-CHAVE

- Escolha **pilha** quando importa a ordem inversa (recursão, backtracking, desfazer).
- Escolha **fila** quando importa a ordem de chegada (escalonamento, BFS).
- Podem ser implementadas sobre vetor (rápido) ou sobre lista encadeada (cresce sob demanda).

5 Deques e Filas de Prioridade (Heaps)

O **deque** (double-ended queue) generaliza pilha e fila: permite inserir e remover em **ambas as extremidades** em O(1). É usado, por exemplo, em algoritmos de janela deslizante e em estruturas que precisam da flexibilidade das duas pontas.

Na **fila de prioridade** os elementos saem por **prioridade**, não por ordem de chegada. A implementação clássica é o **heap binário**: uma árvore binária *completa* em que todo pai respeita uma relação com os filhos — no **min-heap**, o pai é sempre menor ou igual aos filhos (a raiz é o menor elemento); no **max-heap**, o contrário. O heap é guardado de forma compacta em um **vetor**, onde os filhos de `i` ficam em `2i+1` e `2i+2`. Inserção e remoção do topo custam O(log n) porque um elemento "sobe" ou "desce" no máximo a altura da árvore. Heaps são a base do algoritmo de ordenação *heapsort* e do caminho mínimo de Dijkstra.

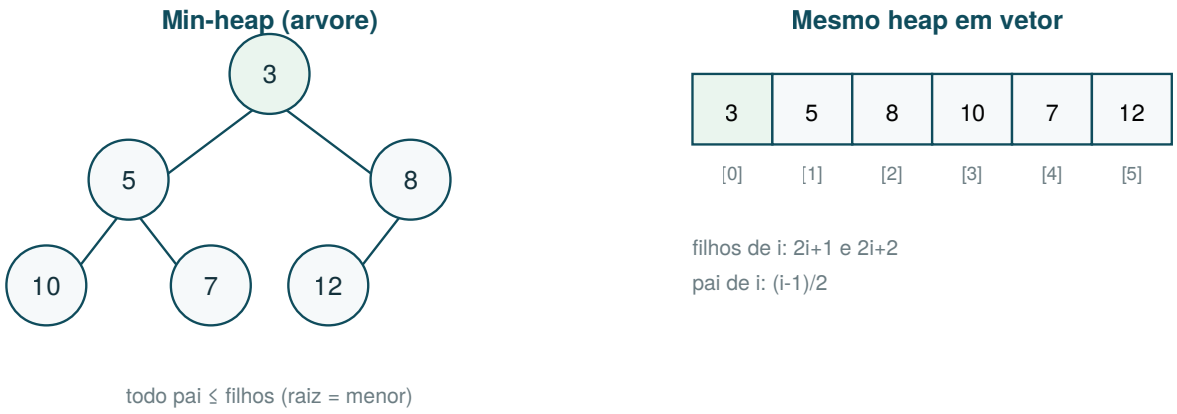


Figura 5 — Min-heap como árvore e sua representação compacta em vetor.

IMPLEMENTACAO EM C

```c
/* MIN-HEAP em vetor: pai(i)=(i-1)/2 ; filhos=2i+1, 2i+2 */
int heap[MAX], n = 0;

void inserir(int x) {                /* O(log n) */
    heap[n] = x;  int i = n++;
    while (i > 0 && heap[(i-1)/2] > heap[i]) {      /* sobe (sift-up) */
        int t = heap[i]; heap[i] = heap[(i-1)/2]; heap[(i-1)/2] = t;
        i = (i-1)/2;
    }
}
int extrair_min(void) {              /* O(log n) */
    int min = heap[0];
    heap[0] = heap[--n];             /* ultimo vai pra raiz */
    int i = 0;
    for (;;) {                       /* desce (sift-down)   */
        int e = 2*i+1, d = 2*i+2, menor = i;
        if (e < n && heap[e] < heap[menor]) menor = e;
        if (d < n && heap[d] < heap[menor]) menor = d;
        if (menor == i) break;
        int t = heap[i]; heap[i] = heap[menor]; heap[menor] = t;
        i = menor;
    }
    return min;
}
```

COMPLEXIDADE

Operacao	Custo	Observacao
Inserir	**O(log n)**	sift-up: sobe até a posição certa
Remover o de maior prioridade	**O(log n)**	sift-down a partir da raiz
Consultar o topo (peek)	**O(1)**	é sempre a raiz do heap
Construir heap a partir de n itens	**O(n)**	heapify de baixo para cima

PONTOS-CHAVE

- Deque = flexibilidade nas duas pontas; fila de prioridade = ordem por importância.
- O heap não é totalmente ordenado — só garante a relação pai/filho.
- Vetor evita ponteiros e mantém tudo contíguo, aproveitando bem a cache.

6 Árvores de Busca (BST) e Balanceadas (AVL / Rubro-Negra)

A **árvore binária de busca (BST)** organiza os dados de modo que, para cada nó, todos os valores da subárvore **esquerda são menores** e os da **direita são maiores**. Essa propriedade permite busca, inserção e remoção em O(h), onde `h` é a altura. Se a árvore está balanceada, `h ≈ log n` e as operações são muito rápidas; mas se os dados entram em ordem, a BST **degenera** em algo parecido com uma lista e cai para O(n).

As **árvores balanceadas** resolvem esse problema mantendo a altura sob controle automaticamente. A **AVL** guarda em cada nó um *fator de balanceamento* (diferença de altura entre as subárvores, que deve ficar em −1, 0 ou +1) e corrige desequilíbrios com **rotações**. A **rubro-negra** colore cada nó de vermelho ou preto e mantém um conjunto de regras (ex.: um nó vermelho não pode ter filho vermelho; todo caminho da raiz às folhas tem o mesmo número de nós pretos) que garantem altura no máximo cerca de 2·log n. Ambas asseguram O(log n) no pior caso; a rubro-negra costuma fazer menos rotações e é a base de `std::map` e de muitos escalonadores.

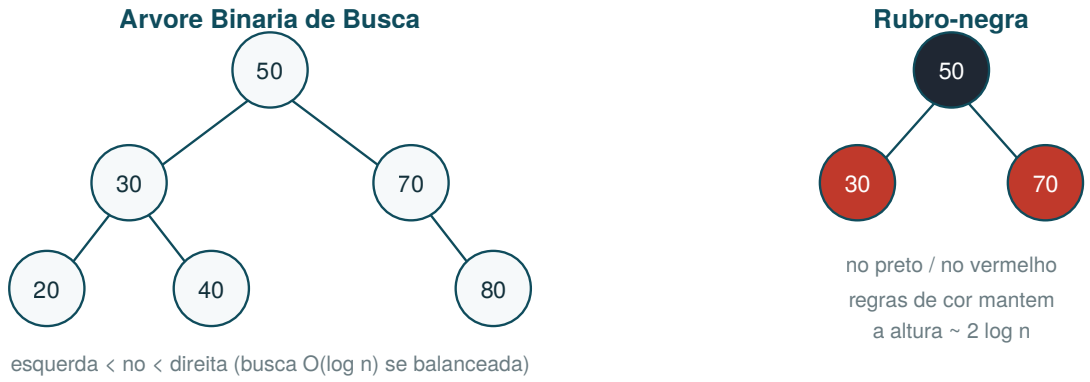


Figura 6 — BST (esq.) e o princípio de coloração da árvore rubro-negra (dir.).

IMPLEMENTACAO EM C

```c
typedef struct No { int dado; struct No *esq, *dir; } No;

/* Insercao em BST — O(h) */
No* inserir(No *raiz, int x) {
    if (raiz == NULL) {
        No *novo = malloc(sizeof(No));
        novo->dado = x; novo->esq = novo->dir = NULL;
        return novo;
    }
    if      (x < raiz->dado) raiz->esq = inserir(raiz->esq, x);
    else if (x > raiz->dado) raiz->dir = inserir(raiz->dir, x);
    return raiz;
}

/* Busca — O(h): O(log n) se balanceada, O(n) se degenerada */
No* buscar(No *raiz, int x) {
    while (raiz && raiz->dado != x)
        raiz = (x < raiz->dado) ? raiz->esq : raiz->dir;
    return raiz;
}
```

COMPLEXIDADE

Operacao	Custo	Observacao
Busca / inserção / remoção (BST)	**O(h)**	h = altura da árvore
BST balanceada	**O(log n)**	caso desejável
BST degenerada	**O(n)**	pior caso — dados em ordem
AVL / Rubro-negra	**O(log n)**	garantido no pior caso via rotações

> **POR QUE BALANCEAR?**
>
> Inserir 1, 2, 3, 4, 5 numa BST simples cria uma "escada" (cada nó só tem filho à direita): a busca vira O(n). As árvores AVL e rubro-negra reorganizam a estrutura com rotações para que isso nunca aconteça, preservando O(log n).

7 Tabela Hash e Tratamento de Colisões

A **tabela hash** busca o "santo graal" das estruturas: acesso em **tempo médio O(1)**. A ideia é usar uma **função hash** que transforma a chave (um nome, um CPF, uma string) em um índice de um vetor. Em vez de procurar sequencialmente, você calcula diretamente onde o dado deveria estar. Uma boa função hash espalha as chaves de forma uniforme e é rápida de calcular.

Como o espaço de chaves é maior que o número de posições, é inevitável que duas chaves diferentes caiam no mesmo índice — isso é uma **colisão**. As duas estratégias clássicas de tratamento são: **encadeamento separado**, em que cada posição guarda uma lista ligada com todas as chaves que colidiram ali; e **endereçamento aberto**, em que se procura a próxima posição livre segundo uma regra de sondagem (linear, quadrática ou hash duplo). O desempenho depende do **fator de carga** (α = nº de elementos ÷ nº de posições): quando ele cresce demais, faz-se um *rehash* (redimensionamento) para manter as operações rápidas.

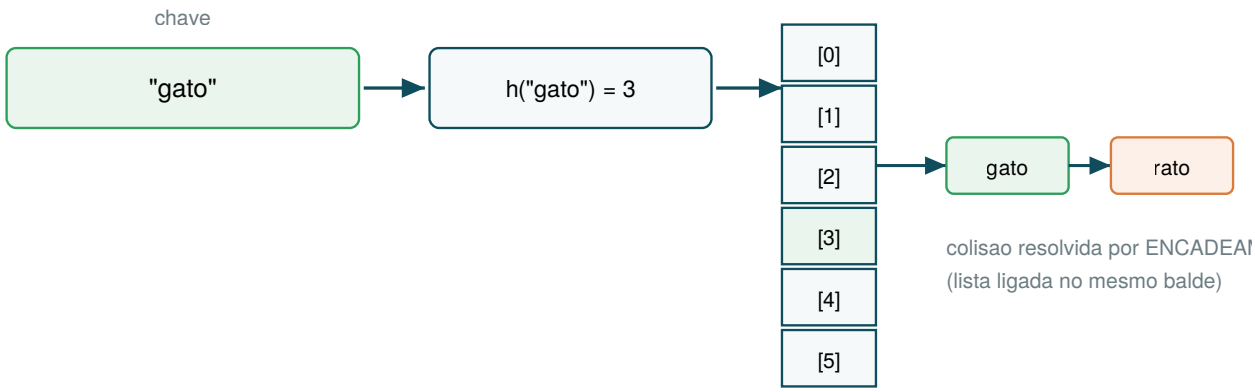


Figura 7 — Função hash mapeando a chave para um balde; colisão resolvida por encadeamento.

IMPLEMENTACAO EM C

```c
#include <string.h>
#define M 101                /* tamanho primo reduz colisoes */

typedef struct No { char *chave; int valor; struct No *prox; } No;
No *tabela[M];

/* funcao hash para strings */
unsigned hash(const char *s) {
    unsigned h = 0;
    while (*s) h = (h * 31 + (unsigned char)*s++) % M;
    return h;
}

/* Insercao com colisao tratada por ENCADEAMENTO — O(1) medio */
void inserir(const char *chave, int valor) {
    unsigned i = hash(chave);
    No *novo = malloc(sizeof(No));
    novo->chave = strdup(chave);
    novo->valor = valor;
    novo->prox  = tabela[i];   /* entra na frente da lista do balde */
    tabela[i]   = novo;
}
```

COMPLEXIDADE

Operacao	Custo	Observacao
Inserção / busca / remoção	**O(1)**	caso médio, com boa função hash
Mesmas operações no pior caso	**O(n)**	muitas colisões no mesmo balde
Rehash (redimensionar)	**O(n)**	recalcula o hash de todos os itens

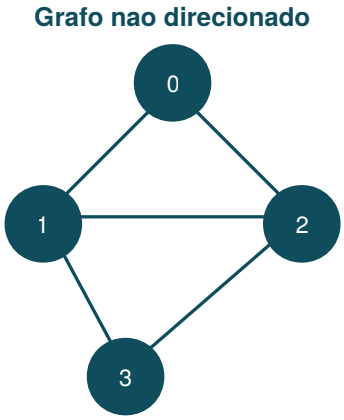
PONTOS-CHAVE

- Não mantém ordem — se você precisa de ordenação, use árvore balanceada.
- A qualidade da função hash e o controle do fator de carga são decisivos.
- Encadeamento é simples e tolera $\alpha > 1$; endereçamento aberto economiza memória mas exige α baixo.

8 Grafos (Matriz e Lista de Adjacência)

Um **grafo** é um conjunto de **vértices** ligados por **arestas**. Ele modela relações: redes sociais, mapas de estradas, dependências entre tarefas, a própria internet. Grafos podem ser **direcionados** (arestas com sentido) ou **não direcionados**, e **ponderados** (arestas com peso/custo) ou não.

Há duas formas principais de representá-los na memória. A **matriz de adjacência** é uma matriz V×V em que a posição `[u][w]` vale 1 (ou o peso) se existe aresta entre `u` e `w`. É simples e verifica a existência de uma aresta em O(1), mas gasta O(V²) de espaço mesmo quando o grafo tem poucas arestas (grafo *esparso*). A **lista de adjacência** guarda, para cada vértice, uma lista dos seus vizinhos. Gasta apenas O(V+E) de espaço e é a escolha natural para grafos esparsos — que são a maioria dos casos reais.



Matriz de adjacencia — O(V²)

	0	1	2	3
0	0	1	1	0
1	1	0	1	1
2	1	1	0	1
3	0	1	1	0

Lista de adjacencia — O(V+E)

0 → 1, 2
1 → 0, 2, 3
2 → 0, 1, 3
3 → 1, 2

Figura 8 — Mesmo grafo nas duas representações: matriz V×V e listas de vizinhos.

IMPLEMENTACAO EM C

```c
#define V 5

/* (1) MATRIZ de adjacencia — O(V^2) de espaco, aresta O(1) */
int adj[V][V] = {0};
void add_matriz(int u, int w) {
    adj[u][w] = adj[w][u] = 1;   /* grafo NAO direcionado */
}

/* (2) LISTA de adjacencia — O(V+E) de espaco */
typedef struct No { int v; struct No *prox; } No;
No *lista[V];
void add_lista(int u, int w) {
    No *a = malloc(sizeof(No)); a->v = w; a->prox = lista[u]; lista[u] = a;
    No *b = malloc(sizeof(No)); b->v = u; b->prox = lista[w]; lista[w] = b;
}
```

COMPLEXIDADE

Operacao	Custo	Observacao
Espaço — matriz	**$O(V^2)$**	boa para grafos densos
Espaço — lista	**O(V+E)**	boa para grafos esparsos
Verificar se existe a aresta (u,w)	**O(1) / O(grau)**	matriz / lista
Percorrer os vizinhos de um vértice	**O(V) / O(grau)**	matriz / lista

PONTOS-CHAVE

- Grafo **denso** (muitas arestas) → matriz; **esparso** (poucas) → lista.
- V = número de vértices, E = número de arestas.
- A representação escolhida impacta diretamente o custo dos algoritmos (BFS, DFS, Dijkstra).

9 Algoritmos de Busca em Grafos (BFS e DFS)

Percorrer um grafo é visitar seus vértices de forma sistemática. As duas travessias fundamentais são a busca em largura e a busca em profundidade; ambas custam **O(V+E)** quando o grafo está em lista de adjacência, e ambas usam um marcador de "visitado" para não repetir vértices.

A **busca em largura (BFS)** explora o grafo **por camadas**: visita todos os vizinhos de um vértice antes de avançar. Ela usa uma **fila** e é a forma natural de encontrar o **menor caminho em número de arestas** num grafo não ponderado. A **busca em profundidade (DFS)** vai **o mais fundo possível** por um caminho antes de voltar (backtracking); usa uma **pilha** (frequentemente a própria recursão). A DFS é a base de detecção de ciclos, ordenação topológica e identificação de componentes conexas.

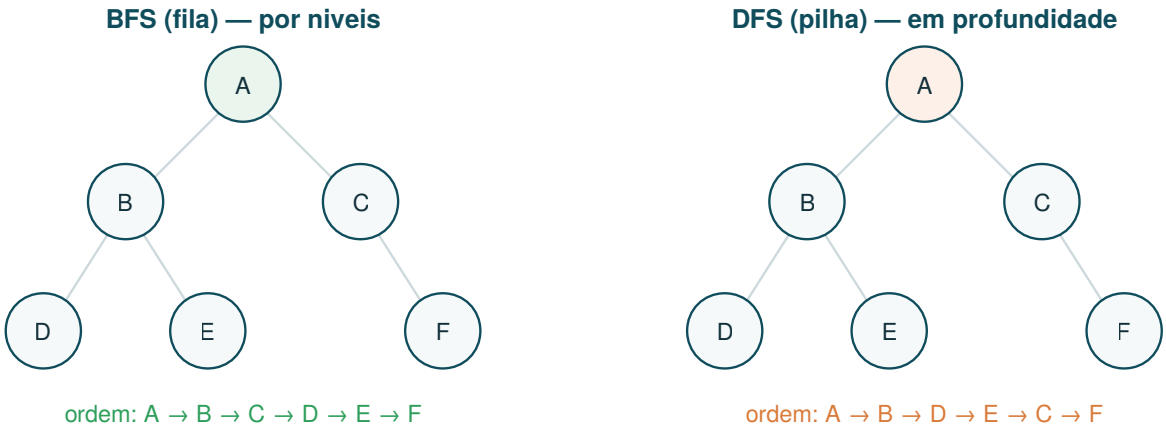


Figura 9 — BFS visita por níveis (fila); DFS mergulha em profundidade (pilha).

IMPLEMENTACAO EM C

```c
/* BFS — usa FILA, visita em largura (por niveis) — O(V+E) */
void bfs(int inicio) {
    int visto[V] = {0}, fila[V], ini = 0, fim = 0;
    visto[inicio] = 1; fila[fim++] = inicio;
    while (ini < fim) {
        int u = fila[ini++];
        printf("%d ", u);
        for (No *p = lista[u]; p; p = p->prox)
            if (!visto[p->v]) { visto[p->v] = 1; fila[fim++] = p->v; }
    }
}

/* DFS — usa PILHA (aqui via recursao) — O(V+E) */
int visto[V];
void dfs(int u) {
    visto[u] = 1;
    printf("%d ", u);
    for (No *p = lista[u]; p; p = p->prox)
        if (!visto[p->v]) dfs(p->v);
}
```

COMPLEXIDADE

Operacao	Custo	Observacao
BFS	**O(V+E)**	usa fila; menor caminho não ponderado
DFS	**O(V+E)**	usa pilha/recursão; ciclos, topológica
Espaço auxiliar	**O(V)**	marcação de visitados + fila/pilha

PONTOS-CHAVE

- **BFS**: caminho mínimo (não ponderado), níveis, alcance mais próximo.
- **DFS**: caminhos, ciclos, ordenação topológica, componentes conexas.
- A diferença essencial é a estrutura auxiliar: fila (BFS) versus pilha (DFS).

Quadro Comparativo

SÍNTESE DAS ESTRUTURAS E SEUS CUSTOS TÍPICOS

Estrutura	Acesso	Busca	Inserção	Remoção	Quando usar
Vetor	**O(1)**	O(n)	O(n)	O(n)	acesso por índice, tamanho estável
Lista encadeada	O(n)	O(n)	**O(1)**	**O(1)**	muitas inserções/remoções
Pilha / Fila	—	—	**O(1)**	**O(1)**	ordem LIFO / FIFO
Heap	O(1) topo	O(n)	O(log n)	O(log n)	prioridade, menor/maior
BST balanceada	O(log n)	**O(log n)**	O(log n)	O(log n)	dados ordenados + busca
Tabela hash	—	**O(1)***	**O(1)***	**O(1)***	busca por chave, sem ordem

* caso médio. Não existe estrutura "melhor" em tudo — a escolha depende sempre das operações mais frequentes do seu problema.

Referencial Teórico

1. CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.
2. ZIVIANI, N. *Projeto de algoritmos: com implementações em Pascal e C*. 3. ed. São Paulo: Cengage Learning, 2011.
3. CELES, W.; CERQUEIRA, R.; RANGEL, J. L. *Introdução a estruturas de dados: com técnicas de programação em C*. Rio de Janeiro: Elsevier, 2004.
4. SZWARCFITER, J. L.; MARKENZON, L. *Estruturas de dados e seus algoritmos*. 3. ed. Rio de Janeiro: LTC, 2010.
5. TENENBAUM, A. M.; LANGSAM, Y.; AUGENSTEIN, M. J. *Estruturas de dados usando C*. São Paulo: Pearson Makron Books, 1995.
6. SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4th ed. Boston: Addison-Wesley, 2011.
7. FEOFILOFF, P. *Algoritmos em linguagem C*. Rio de Janeiro: Elsevier, 2009.